RTVRPTW | ExpositoTOP

None

Daniel Alejandro Álvarez Casablanca

Práctica 3 | Curso 2026-2027

Índice

1. RTVRPTW · ExpositoTOP 3
   1.1 Componentes 3
2. Introducción y alcance 4
   2.1 Problema que se resuelve 4
   2.2 Origen y límites del código 4
   2.3 Organización de los ficheros 4
3. Arquitectura del sistema 5
   3.1 Módulo principal `top` 5
   3.2 Flujo de ejecución 5
   3.3 Datos de las instancias 5
4. Manual de usuario 6
   4.1 Requisitos 6
   4.2 Ejecución rápida 6
   4.3 Ejecución controlada 6
   4.4 Resolución de problemas 6
5. Maven · Construcción del proyecto 7
   5.1 POM y convención de directorios 7
   5.2 Fases del ciclo de vida 7
   5.3 Plugins configurados 7
   5.4 Uso desde Cursor 7
6. Calidad de código · SonarQube 8
   6.1 Configuración 8
   6.2 Hallazgos al inspeccionar el código original 8
   6.3 Evidencias reales pendientes de completar 8
   6.4 Métricas vs. observaciones 8
   6.5 Análisis real inicial de SonarQube (01/11/2026) 9
7. Documentación automática con Doxygen 10
   7.1 Comentarios 10
   7.2 Doxyfile y Graphviz 10
   7.3 Integración con MkDocs 10
8. Decisiones técnicas 11
   8.1 ADR-001 · Adoptar Maven 11
   8.2 ADR-002 · Manejar errores sin terminar la JVM 12

1. RTVRPTW · ExpositoTOP

Portal de documentación técnica · Práctica 3 · Curso 2026–2027

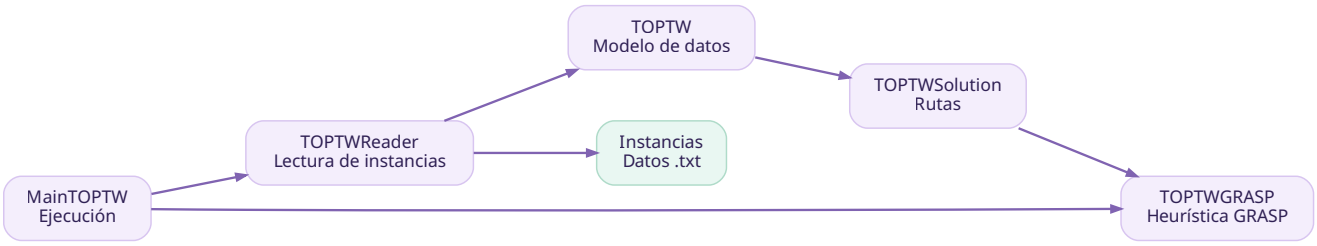
Universidad de La Laguna · Grado en Ingeniería Informática · Laboratorio de Desarrollo y Herramientas

Este portal reúne la documentación de un recomendador turístico con ventanas de tiempo (*Team Orienteering Problem with Time Windows*, TOPTW), adaptado desde el proyecto `ExpositoTOP.rar`.

> **Qué contiene**
>
> Código Java reorganizado para Maven, pruebas con JUnit 5, integración de SonarQube y JaCoCo, comentarios Doxygen, grafos Graphviz y páginas Docs-as-Code con MkDocs y Material.

1.1 Componentes

Herramienta	Papel en el proyecto
Maven	Compilar, probar, empaquetar, instalar y desplegar un artefacto local
JUnit 5 + JaCoCo	Probar funcionalidades y generar cobertura medible
SonarQube	Evaluar mantenibilidad, fiabilidad y seguridad del código
Doxygen + Graphviz	Extraer documentación de las clases y producir diagramas
MkDocs + Material + MkDoxy	Publicar el portal unificado de documentación



Primer paso: consulta el manual de usuario o la guía `GUIA_PASO_A_PASO.md` incluida en el repositorio.

Autor del informe: Daniel Alejandro Álvarez Casablanca.

Transparencia: las métricas numéricas de SonarQube se completarán al ejecutar el servidor y conservar las evidencias reales. Ninguna métrica se ha inventado.

2. Introducción y alcance

2.1 Problema que se resuelve

RTVRPTW, en el archivo original denominado `ExpositoTOP`, aborda la construcción de itinerarios turísticos que deben respetar ventanas temporales. Cada punto tiene coordenadas, una recompensa, un tiempo de servicio y una hora mínima/máxima de visita. La solución intenta seleccionar y ordenar visitas en varias rutas factibles.

La heurística `TOPTWGRASP` emplea listas restringidas de candidatos (RCL) y selección aleatoria para construir rutas. El programa original posee un lazo de 10 000 iteraciones para cada uno de varios tamaños de RCL y 29 instancias, que puede generar mucho texto en consola.

2.2 Origen y límites del código

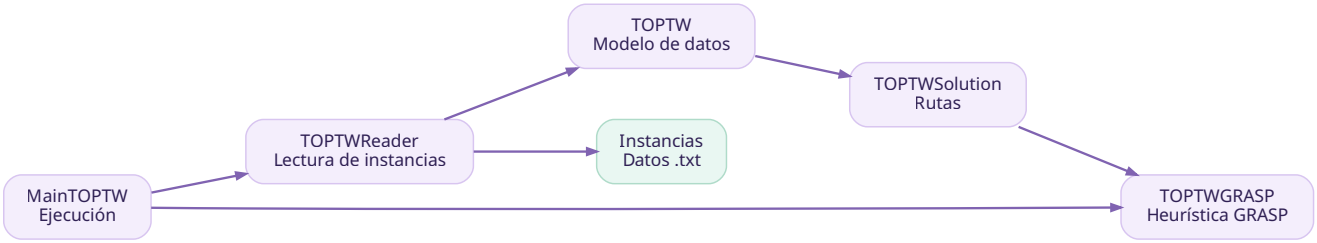
El código y las instancias proceden exclusivamente del archivo `ExpositoTOP.rar` proporcionado con la práctica. No se afirma que la búsqueda local esté implementada: su bloque aparece como comentario en `TOPTWGRASP` y el evaluador `TOPTWEvaluator.evaluate()` es un método cuyo cuerpo efectivo está vacío.

La migración mantiene el algoritmo de construcción. Las modificaciones se centran en empaquetado, entradas de ejecución, control de errores, documentación y pruebas.

2.3 Organización de los ficheros

```
src/main/java/       Clases Java originales adaptadas
src/main/resources/  Conjunto de instancias de los problemas
src/test/java/       Pruebas unitarias nuevas
pom.xml              Ciclo de vida y plugins de Maven
Doxyfile             Configuración de Doxygen y Graphviz
mkdocs.yml           Portal MkDocs / MkDoxy / Material
scripts/             Automatización de comprobaciones
```

3. Arquitectura del sistema



3.1 Módulo principal `top`

Clase	Responsabilidad principal
`MainTOPTW`	Selecciona instancias y lanza el algoritmo GRASP
`TOPTWReader`	Lee una instancia desde recursos del JAR o disco y crea el problema
`TOPTW`	Conserva puntos, ventanas temporales, puntuaciones y distancias
`TOPTWSolution`	Representa las rutas mediante sucesores y predecesores
`TOPTWGRASP`	Construye soluciones voraces aleatorizadas con RCL
`TOPTWEvaluator`	Marcador de evaluación; implementación sin completar
`TOPTWRoute`	Estructura auxiliar para rutas

Las clases de `es.ull.esit.utilities` y `es.ull.esit.utils` ofrecen funciones de formato, matrices, recorridos y pares. Los diagramas UML detallados se obtienen ejecutando `doxygen Doxyfile`, con `HAVE_DOT = YES`.

3.2 Flujo de ejecución

1. `MainTOPTW` elige una instancia.
2. `TOPTWReader` carga el fichero.
3. `TOPTW` construye la matriz de distancias.
4. `TOPTWSolution` prepara las rutas.
5. `TOPTWGRASP` construye rutas candidatas y comunica sus resultados.

3.3 Datos de las instancias

El directorio de recursos contiene datos de prueba `c*`, `r*`, `rc*`, `pr*` y `c101Recortada.txt` y un documento `formatInstances.txt` con su especificación. Para pruebas rápidas se utiliza la instancia recortada.

4. Manual de usuario

4.1 Requisitos

JDK 17 o superior; Apache Maven 3.9+ recomendado. Para generar documentación: Python 3.11+, Doxygen, Graphviz y los módulos de `requirements-docs.txt`.

4.2 Ejecución rápida

Desde la raíz del proyecto:

```
mvn clean verify
java -jar target/expositotop-1.0.0-SNAPSHOT.jar --quick
```

`--quick` selecciona una instancia reducida, una iteración y RCL de tamaño 3. La ejecución normal (`java -jar ...` sin argumentos) conserva las 29 instancias y los ciclos largos originales.

4.3 Ejecución controlada

```
java -jar target/expositotop-1.0.0-SNAPSHOT.jar --instance c101.txt --iterations 2
```

Parámetros disponibles:

Argumento	Descripción
`--quick`	Demostración con `c101Recortada.txt`
`--instance NOMBRE`	Nombre de una instancia incluida en `Instances/TOPTW/`
`--iterations N`	Número positivo de iteraciones por RCL

Aviso: las soluciones pueden variar entre ejecuciones porque la selección incluye aleatoriedad; la puntuación mostrada no es una métrica de SonarQube.

4.4 Resolución de problemas

- **`mvn: command not found`:** instala Maven y vuelve a abrir la terminal.
- **`java: command not found`:** verifica el JDK con `java -version`.
- **No se encuentra la instancia:** comprueba el nombre exacto y que el recurso se empaquetó dentro de `target/*.jar`.
- **La ejecución produce demasiado texto:** utiliza `--quick` y pocas iteraciones.

5. Maven · Construcción del proyecto

5.1 POM y convención de directorios

El archivo `pom.xml` declara `groupId=es.ull.esit`, `artifactId=expositotop`, versión `1.0.0-SNAPSHOT`, empaquetado JAR y Java 17. Maven utiliza la estructura `src/main/java`, `src/main/resources` y `src/test/java`.

5.2 Fases del ciclo de vida

Fase	Comando	Qué comprueba
validate	`mvn validate`	Sintaxis y metadatos del proyecto
compile	`mvn compile`	Compilación del código fuente
test	`mvn test`	Ejecución de las pruebas JUnit 5
package	`mvn package`	Generación del JAR en `target/`
install	`mvn install`	Copia al repositorio Maven local `~/.m2`
deploy	`mvn deploy`	Copia a `.artifacts/maven-repository` local; no publica en Internet

Importante: al ejecutar una fase, Maven ejecuta las anteriores del mismo ciclo. Para análisis de cobertura usa `mvn clean verify`, que ejecuta JaCoCo en la fase `verify`.

5.3 Plugins configurados

Plugin	Finalidad
`maven-compiler-plugin`	Compilar Java 17
`maven-surefire-plugin`	Ejecutar JUnit 5
`maven-jar-plugin`	Establecer la clase principal del JAR
`exec-maven-plugin`	Ejecutar `MainTOPTW` sin llamar directamente a Java
`jacoco-maven-plugin`	Instrumentar pruebas y generar XML para SonarQube
`sonar-maven-plugin`	Enviar el análisis al servidor SonarQube

5.4 Uso desde Cursor

Abre la carpeta que contiene `pom.xml`, instala **Extension Pack for Java** y **Maven for Java** si falta. En el panel *Maven*, despliega el proyecto y ejecuta `Lifecycle > validate`, `compile`, `test`, `package`, `install`, `deploy`. También puedes abrir la terminal integrada y ejecutar los comandos anteriores. Registra capturas de las dos modalidades para la entrega.

6. Calidad de código · SonarQube

6.1 Configuración

SonarQube se usa con el objetivo de identificar problemas de fiabilidad, seguridad y mantenibilidad. El POM fija `sonar.projectKey`, URL local `http://localhost:9000` y la ruta de cobertura XML de JaCoCo.

```
export SONAR_TOKEN="TU_TOKEN_LOCAL"
./scripts/sonar.sh
```

El script ejecuta `mvn clean verify` y el objetivo versionado de SonarScanner for Maven. El servidor debe estar activo antes del análisis.

6.2 Hallazgos al inspeccionar el código original

Estos son **hallazgos manuales comprobados sobre los archivos originales**, NO incidencias ni métricas certificadas por un informe de SonarQube.

Archivo	Hallazgo	Solución propuesta o realizada
`TOPTWReader.java`	`System.exit(0)` durante un fallo de lectura	**Corregido:** excepción informativa y cierre automático
`ExpositoUtilities.java`	`reader.close()` cuando el lector puede ser `null`	**Corregido:** `try-with-resources`
`mainTOPTW.java`	Nombre de clase sin convención y pruebas extensas por defecto	**Corregido:** `MainTOPTW` y opción `--quick`
`TOPTWEvaluator.java`	Evaluación comentada y método sin efectos	**Pendiente:** implementar la evaluación y probarla
`TOPTWGRASP.java`	Bloque de búsqueda local comentado y gran volumen de salida	**Pendiente:** implementar búsqueda local y logging configurable
`TOPTWSolution.java`	Método `setWaitingTime(int,int)` reduce la precisión del campo `double[]`	**Pendiente:** aceptar `double` y cubrir con pruebas

6.3 Evidencias reales pendientes de completar

Después de ejecutar SonarQube, anota en `evidencias/RESULTADOS_SONAR.md`: versión, fecha, Quality Gate, bugs/issues, vulnerabilidades, *code smells*, duplicación, deuda técnica, cobertura, capturas del panel y, si es posible, reglas/identificadores. Sin esas capturas no debe declararse el análisis como concluido.

6.4 Métricas vs. observaciones

Una compilación correcta prueba que el código Java compila; **no prueba** ausencia de errores lógicos ni conformidad con el Quality Gate. JaCoCo mide cobertura de pruebas, que no implica calidad funcional. SonarQube es análisis estático y no reemplaza pruebas ni revisión humana.

6.5 Análisis real inicial de SonarQube (01/11/2026)

Esta primera medición se obtuvo **antes** de aplicar la revisión de código.

- Quality Gate: **PASSED**
- Incidencias abiertas: **5** (fiabilidad: 1 bloqueante, 1 alta y 3 medias).
- Seguridad: **0** incidencias.
- Duplicación: **0,0 %**.
- Cobertura de pruebas: **12,6 %**.
- Código analizado: aproximadamente **1,2 K líneas**.
- Esfuerzo total estimado para solucionar incidencias: **35 minutos**.

Las cinco incidencias se atribuyen a: cierre inseguro de BufferedWriter, creación repetida de Random, contrato de iteración de PowerSet, resultado de readLine sin comprobar, y sobrecarga confusa de equals(TOPTWSolution).

Validación pendiente: volver a ejecutar `mvn clean verify` y el análisis de SonarQube en Windows tras estas correcciones. Hasta entonces no hay una segunda medición contrastada y no debe declararse una reducción real de incidencias.

7. Documentación automática con Doxygen

7.1 Comentarios

Las clases principales incluyen comentarios compatibles con Doxygen `\brief` y descripciones detalladas. Se comentaron operaciones fundamentales de lectura, construcción de rutas y distancias. El código original no documentaba sistemáticamente todas las clases y métodos.

Ejemplo incluido en `TOPTWReader.java`:

```
/**
 * \brief Construye un problema TOPTW a partir de un fichero de instancia.
 * \param filePath Ruta lógica del recurso o archivo local.
 * \return Problema con matriz de distancias calculada.
 */
public static TOPTW readProblem(String filePath) { /* ... */ }
```

7.2 Doxyfile y Graphviz

Configuraciones principales:

```
INPUT = src/main/java
RECURSIVE = YES
FILE_PATTERNS = *.java
OPTIMIZE_OUTPUT_JAVA = YES
HAVE_DOT = YES
DOT_IMAGE_FORMAT = svg
GENERATE_HTML = YES
GENERATE_LATEX = YES
```

Generación:

```
mkdir -p .artifacts
doxygen Doxyfile
open .artifacts/doxygen/html/index.html
```

Graphviz proporciona diagramas de colaboración y herencia cuando la estructura del código permite dibujarlos. El número de grafos depende de las relaciones reales entre clases.

7.3 Integración con MkDocs

El plugin **MkDoxy** se configura en `mkdocs.yml` para extraer automáticamente la referencia de la API Java. MkDocs utiliza el tema **Material**, y el plugin `with-pdf` genera un PDF técnico desde las páginas Markdown.

```
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -r requirements-docs.txt
./scripts/docs.sh
```

El HTML de Doxygen se genera en `.artifacts/doxygen/html/` y el portal en `site/`. Comprueba que `site/pdf/documentacion-tecnica.pdf` exista tras construir el portal.

8. Decisiones técnicas

8.1 ADR-001 · Adoptar Maven

Estado: aceptada para la práctica.

8.1.1 Contexto

El proyecto original se entregó en un `.rar` con configuración de Ant/NetBeans y sin un `pom.xml` utilizable como proyecto Maven estándar.

8.1.2 Decisión

Mover los `.java` a `src/main/java`, integrar las instancias en `src/main/resources`, añadir pruebas en `src/test/java` y versionar las fases y plugins desde un único `pom.xml`.

8.1.3 Consecuencias

- Positivas: estructura uniforme, IDE, dependencias declarativas, empaquetado reproducible y conexión directa con SonarScanner for Maven.
- Costes: descargar plugins y dependencias al primer uso; requiere migrar las rutas de lectura.

8.1.4 Alternativas

Conservar Ant, o usar Gradle. No se eligieron porque la práctica exige experimentar explícitamente con las fases de Maven.

8.2 ADR-002 · Manejar errores sin terminar la JVM

Estado: aceptada e implementada parcialmente.

8.2.1 Contexto

En `TOPTWReader.java`, el código original imprimía la excepción y ejecutaba `System.exit(0)` ante problemas de lectura. El cero representa habitualmente una terminación correcta y dificulta la integración con pruebas, IDE y automatización.

8.2.2 Decisión

Usar `try-with-resources` y propagar una excepción `IllegalArgumentException` con la ruta de entrada. Priorizar recursos del classpath para que las instancias estén disponibles en el JAR.

8.2.3 Consecuencias

El consumidor puede decidir qué hacer ante el error sin terminar forzosamente el proceso. Se han añadido pruebas para rutas inexistentes. El formato de instancias aún necesita una validación de esquema más exhaustiva.